Contents

1. Objectives

The primary objectives of this project were:

- To develop an interactive **maze-based escape game** using OpenGL and C++.
- To implement **real-time player movement** with intuitive keyboard controls.
- To design a **collision detection system** to prevent the player from passing through walls or obstacles.
- To create a **dynamic difficulty system** with varying maze sizes and time constraints.
- To enhance our understanding of **computer graphics principles**, including rendering, transformations, and user interaction.

2. Introduction

Maze Escape is a 3D puzzle game that challenges players to navigate through a procedurally generated maze and reach the exit before time runs out. Built using **OpenGL** and **C++**, this project showcases:

- **Procedural maze generation** uses recursive backtracking.
- **Collision detection** to ensure realistic movement and interactions.
- **A timer-based challenge** with adjustable difficulty levels.
- **3D rendering** of game elements, including walls, the player character, obstacles, and the exit.

This game serves as both an **educational tool** for learning computer graphics and a **foundation** for more complex game development projects.

3. Application

The project has several practical applications:

- **Education**: Demonstrates core OpenGL concepts such as rendering, transformations, and user input handling.
- **Game Development**: Acts as a prototype for more advanced games, such as dungeon crawlers or puzzle-based adventures.
- **Algorithm Testing**: Provides a framework for experimenting with pathfinding algorithms like BFS, DFS, or A*.
- **Graphics Practice**: Offers hands-on experience with 3D rendering and collision detection systems.

3. System Architecture

The Maze Escape game follows a modular architecture with three core subsystems:

1. **Maze Generator**
    - Uses recursive backtracking to build solvable mazes
    - Places obstacles strategically using BFS pathfinding
    - Adjusts size based on difficulty (11x11 to 21x21)

2. **Player Input**
    - Handles keyboard inputs (WASD/Arrows)
    - Manages collisions with walls/obstacles
    - Tracks game states (Start/Playing/Won/Lost)
    - Runs countdown timer

3. **Rendering Engine**
    - 3D visualization using OpenGL
    - Top-down camera view
    - Draws maze walls, player sphere, exit cube
    - Displays UI (timer, messages)

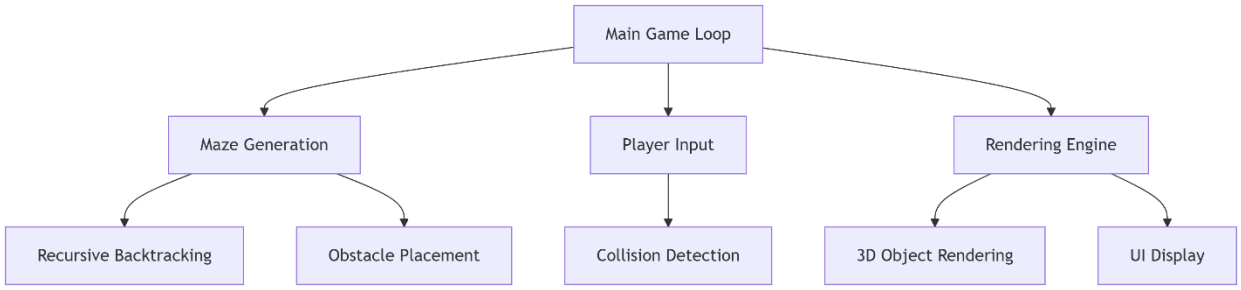


Figure 1: System Architecture

5. Features

5.1 Core Mechanics

- **Procedural Maze Generation**:
  - The maze is generated using **recursive backtracking**, ensuring a unique layout each time.
  - Random loops are added to increase complexity and variety.

- **Player Controls**:
  - Intuitive **WASD or Arrow Key** controls for smooth navigation.
  - Real-time **collision detection** to prevent wall and obstacle penetration.

- **Obstacles**:
  - Red cubes are strategically placed to block paths without making the maze unsolvable.

- **Timer System**:
  - A countdown timer adds urgency, with **60 seconds for Easy**, **45 for Medium**, and **30 for Hard** difficulty.
  - The game ends if time expires.

5.2 Visual Elements

Element	Rendering Method	Description
Walls	glutSolidCube ( )	Gray, block-like structures forming the maze boundaries.
Player	glutSolidSphere ( )	A blue sphere representing the player character.
Exit	glutSolidCube ( )	A yellow cube marking the end goal.
Obstacles	glutSolidCube ( )	Red cubes placed to block certain paths.
Ground	Grid of quads	Dark green surface providing a visual base for the maze.

5.3 User Interface

- **Start Screen**: Allows players to select difficulty (Easy, Medium, Hard).
- **In-Game HUD**: Displays the remaining time.
- **End Screens**: Shows "You Won!" or "Time's Up!" messages upon game completion.

6. Workflow

The game follows a straightforward workflow through four key states. It begins in the START state where players select difficulty (Easy/Medium/Hard). Upon selection, it transitions to the PLAYING state - generating a maze, placing the player at the start position, and starting the countdown timer. During gameplay, the system continuously checks for two outcomes: if the player reaches the exit, it transitions to the WON state; if time expires or the player crashes, it moves to the LOST state. Both WON and LOST states display appropriate messages and wait for the player to press 'R', which resets the game back to the START state. This clean, looped structure ensures intuitive gameplay with clear win/lose conditions and instant replayability.

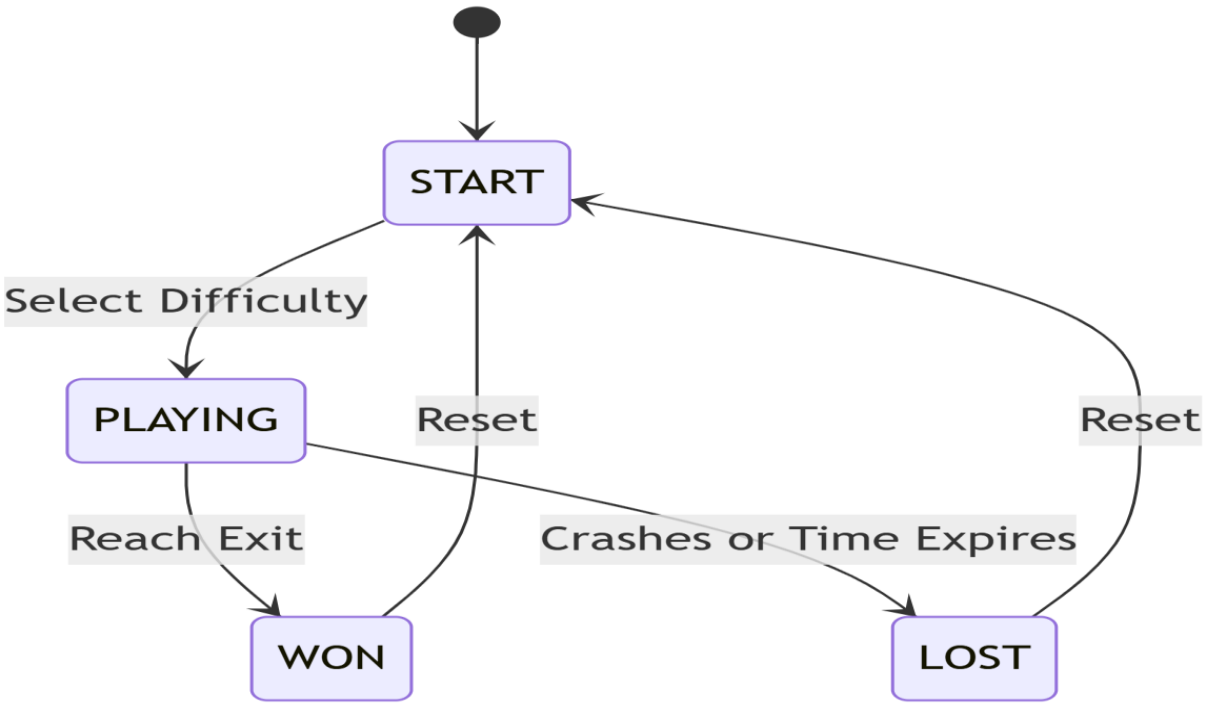


Figure 2: Workflow Diagram

7. Implementation & Code Walkthrough

This section provides a comprehensive technical breakdown of our Maze Escape game, combining system architecture with detailed code analysis. We'll examine each major component through both conceptual and implementation lenses.

7.1 Maze Generation System

The maze generation employs a recursive backtracking algorithm that:

- Guarantees a solvable path from start to finish
- Creates organic, non-uniform maze layouts
- Supports difficulty scaling through size parameters
- Implements random loops for increased complexity

Code Implementation:

```
void carveMaze(int x, int y, vector<vector<bool>>& visited) {
    const int dx[] = {1, -1, 0, 0};  // Direction vectors
    const int dy[] = {0, 0, 1, -1};

    // Randomize direction selection
    vector<int> directions = {0, 1, 2, 3};
    shuffle(directions.begin(), directions.end(), rng);

    for (int dir : directions) {
        int nx = x + dx[dir] * 2;  // Move two cells
        int ny = y + dy[dir] * 2;

        if (nx > 0 && nx < mazeWidth-1 && ny > 0 && ny < mazeHeight-1) {
            if (!visited[ny][nx]) {
                visited[ny][nx] = true;
                maze[y + dy[dir]][x + dx[dir]] = 0; // Break wall
                maze[ny][nx] = 0; // Create new cell
                carveMaze(nx, ny, visited);
            }
            else if (rng() % 100 < 25) { // 25% chance to create loop
                maze[y + dy[dir]][x + dx[dir]] = 0;
            }
        }
    }
}
```

Key Features:

- Uses Mersenne Twister (`mt19937`) for high-quality randomness
- Maintains border walls by staying within (1,1) to (width-2,height-2)
- Visualized wall-breaking by setting intermediate cells to 0

7.2 Collision & Movement System

The movement system provides:

- Four-directional navigation (WASD/Arrows)

- Real-time collision detection
- Prevention of wall/obstacle penetration
- Immediate feedback for player actions

Code Implementation:

```
bool isValidMove(int x, int y) {
    // Boundary check - prevents out-of-maze movement
    if (x < 0 || x >= mazeWidth || y < 0 || y >= mazeHeight)
        return false;

    // Wall collision - checks maze grid value
    if (maze[y][x] == 1)
        return false;

    // Obstacle check - verifies against obstacle positions
    for (auto& obs : obstacles) {
        if (obs.first == x && obs.second == y)
            return false;
    }

    return true; // Only returns true for valid path cells
}

void movePlayer(int dx, int dy) {
    int newX = playerX + dx;
    int newY = playerY + dy;

    if (isValidMove(newX, newY)) {
        playerX = newX;
        playerY = newY;

        // Win condition check
        if (newX == exitX && newY == exitY) {
            gameState = WON;
        }
    }
}
```

Optimizations:

- Early termination in obstacle checks
- Combined boundary/wall checks for efficiency
- Immediate state update on win condition

7.3 Obstacle Placement System

The obstacle system ensures:

- Guaranteed solvability despite obstacles
- Strategic placement away from critical paths
- Difficulty-based quantity scaling
- Visual distinction from walls

Code Implementation:

```
void addObstacles() {
    // Find guaranteed solution path
    auto solutionPath = findShortestPath();
    vector<vector<bool>> protectedCells(mazeHeight, vector<bool>(mazeWidth, false));

    // Mark critical path cells
    for (auto& p : solutionPath) {
        protectedCells[p.second][p.first] = true;
    }

    // Determine obstacle count by difficulty
    int targetCount;
    switch(difficulty) {
        case EASY: targetCount = 5; break;
        case MEDIUM: targetCount = 10; break;
        case HARD: targetCount = 15; break;
    }

    // Place obstacles randomly in non-critical areas
    while (obstacles.size() < targetCount) {
        int x = rng() % mazeWidth;
        int y = rng() % mazeHeight;
```

```
            if (!protectedCells[y][x] &&       // Not on solution path
                maze[y][x] == 0 &&             // Not a wall
                !(x == playerX && y == playerY)) {  // Not on player

                obstacles.emplace_back(x, y);
            }
        }
    }
```

Path Preservation:

- Uses BFS to identify critical path
- Marks those cells as protected
- Only places obstacles in non-protected path cells
- Verifies empty path cell before placement

7.4 Rendering Pipeline

The rendering system features:

- 3D perspective with dynamic camera
- Layered drawing (ground → walls → objects → UI)
- Optimized matrix operations
- Clean visual hierarchy

Code Implementation:

```
void display() {
    // Clear buffers and reset view
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    // Camera setup - top-down perspective
    gluLookAt(mazeWidth/2.0f, mazeHeight*1.5f, mazeHeight/2.0f,
              mazeWidth/2.0f, 0.0f, -mazeHeight/2.0f,
              0.0f, 1.0f, 0.0f);

    // Draw ground plane
    glBegin(GL_QUADS);
    glColor3f(0.2f, 0.4f, 0.2f); // Dark green
```

```
    glVertex3f(0, -0.5f, 0);
    glVertex3f(mazeWidth, -0.5f, 0);
    glVertex3f(mazeWidth, -0.5f, -mazeHeight);
    glVertex3f(0, -0.5f, -mazeHeight);
    glEnd();

    // Draw walls and objects
    for (int y = 0; y < mazeHeight; y++) {
        for (int x = 0; x < mazeWidth; x++) {
            if (maze[y][x] == 1) {
                drawWall(x, y); // Gray cubes
            }
        }
    }

    // Interactive elements
    drawExit(exitX, exitY); // Yellow cube
    drawPlayer(playerX, playerY); // Blue sphere

    for (auto& obs : obstacles) {
        drawObstacle(obs.first, obs.second); // Red cubes
    }

    // UI elements
    renderTimer();
    if (gameState != PLAYING) renderGameStateMessage();

    glutSwapBuffers();
}
```

Rendering Optimizations:

- Batch drawing of ground plane
- Selective wall rendering (only where maze[y][x] == 1)
- UI drawn last to ensure visibility
- Matrix stack management with glPush/PopMatrix

7.5 Game State Management

The state system handles:

- Game phase transitions (start → play → win/lose)
- Difficulty selection
- Timer management
- Reset functionality

Code Implementation:

```
// State transition handler
void keyboard(unsigned char key, int, int) {
  switch(key) {
    // Difficulty selection
    case '1': difficulty = EASY; initGame(); break;
    case '2': difficulty = MEDIUM; initGame(); break;
    case '3': difficulty = HARD; initGame(); break;

    // Reset functionality
    case 'r': case 'R':
      gameState = START;
      break;

    // Movement controls
    case 'w': movePlayer(0, 1); break;
    case 's': movePlayer(0, -1); break;
    case 'a': movePlayer(-1, 0); break;
    case 'd': movePlayer(1, 0); break;
  }
  glutPostRedisplay();
}

// Timer callback
```

```
void timer(int) {
  if (gameState == PLAYING) {
    clock_t current = clock();
    timeRemaining -= float(current - lastTime)/CLOCKS_PER_SEC;
    lastTime = current;

    if (timeRemaining <= 0) {
      timeRemaining = 0;
      gameState = LOST;
    }
  }
  glutPostRedisplay();
  glutTimerFunc(1000, timer, 0); // Recursive 1-second timer
}
```

State Machine Logic:

- START: Shows menu, awaits difficulty selection
- PLAYING: Active game with timer countdown
- WON/LOST: Displays outcome, awaits reset
- Uses glutTimerFunc for smooth timing
- Maintains frame independence for movement

This approach demonstrates how each conceptual element translates directly into implemented code, showing the complete technical execution of our Maze Escape game.

8. Visual Walkthrough

8.1 Front Screen Elements

- Title *"MAZE ESCAPE"* displayed in bold, centered, and spaced lettering.

- Three rounded purple buttons for difficulty selection: EASY (60s), MEDIUM (45s), HARD (30s).
- Movement and objective instructions shown at the bottom of the screen.

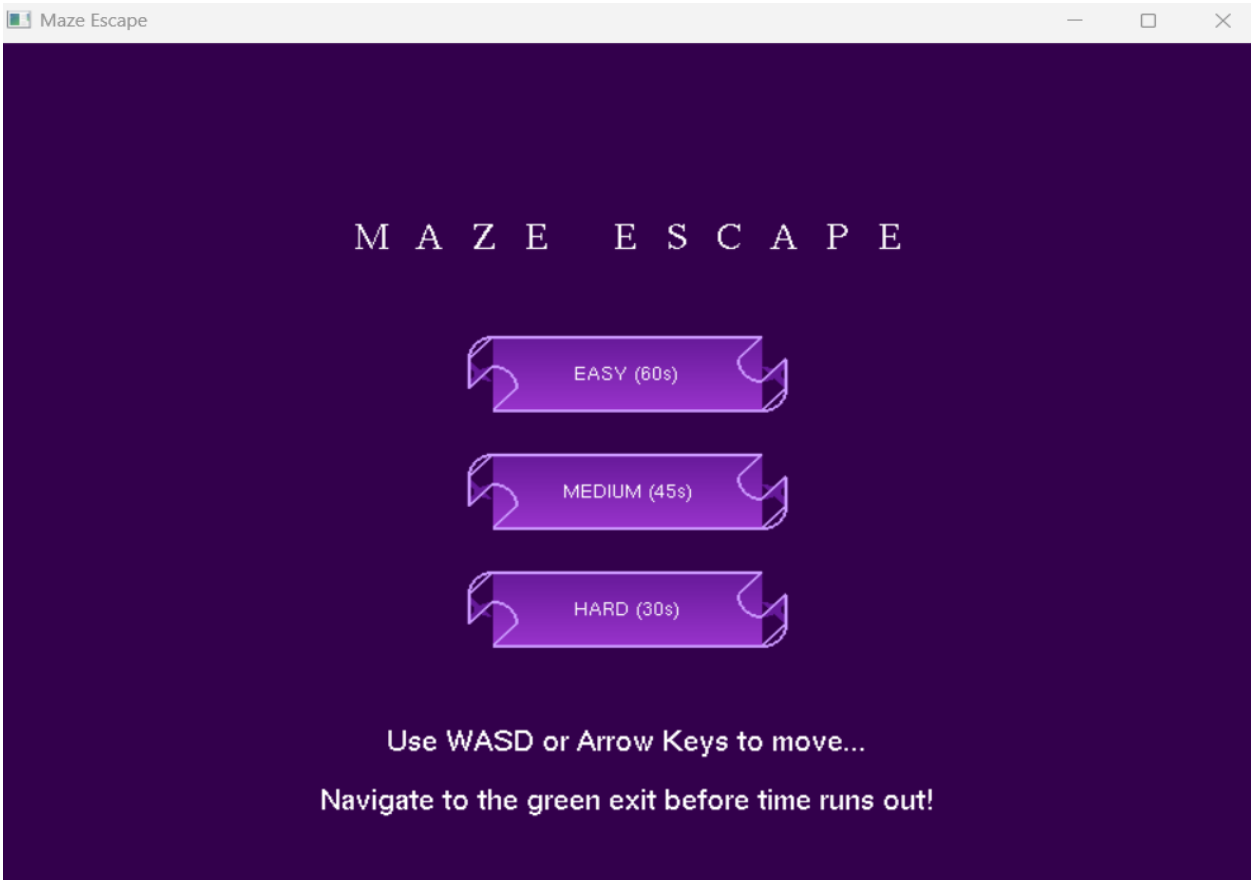
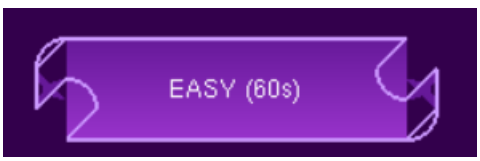


Figure 3: Front Screen (Home)

8.2 Easy Level Maze

- Maze size: 11×11 grid with wider, simpler paths.
- Contains 5 red obstacles positioned away from the shortest path.
- Time limit: 60 seconds to reach the green exit cube.



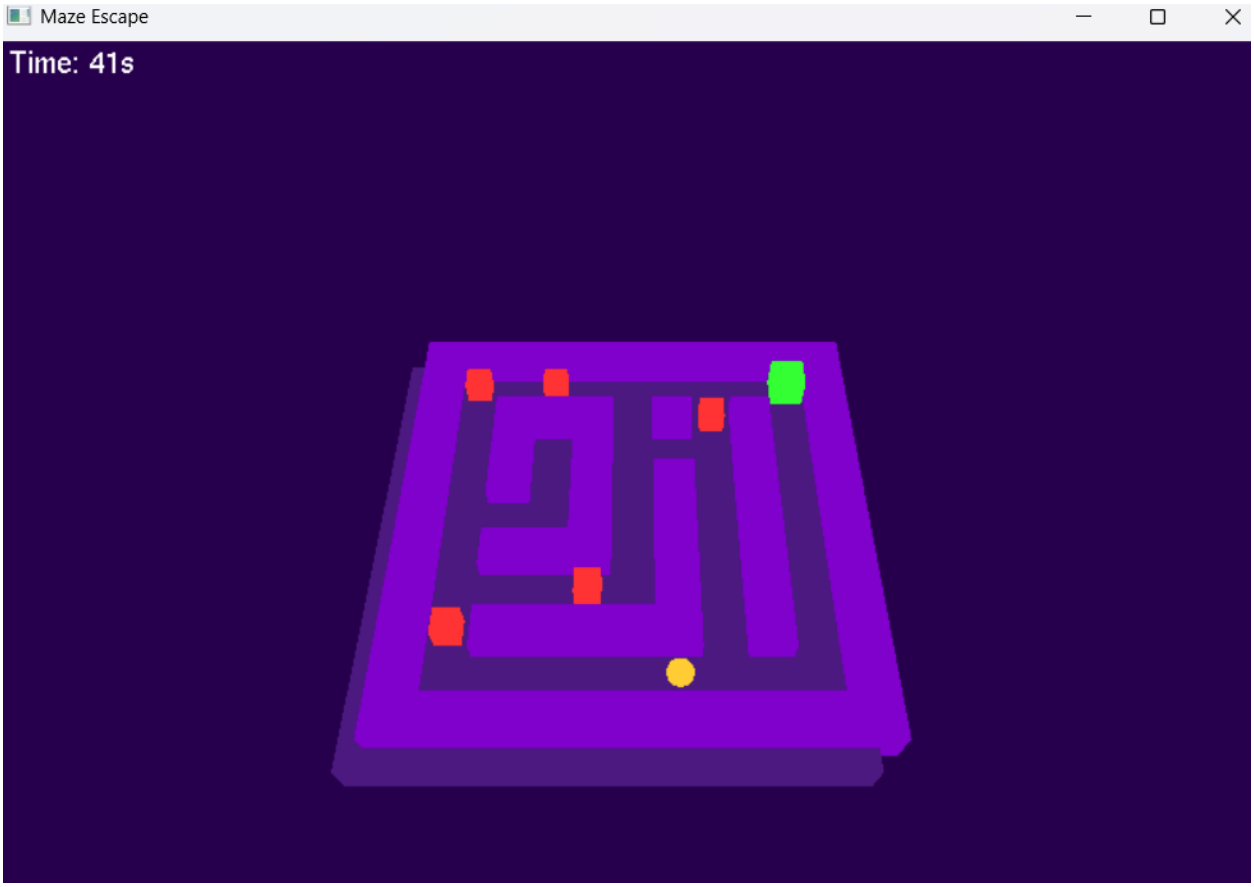
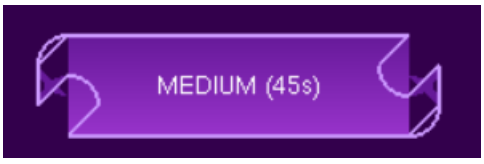


Figure 4: Level - Easy

8.3 Medium Level Maze

- Maze size: 15×15 grid with narrower and more complex passages.
- Contains 10 red obstacles placed to partially block potential routes.
- Time limit: 45 seconds to reach the green exit cube.



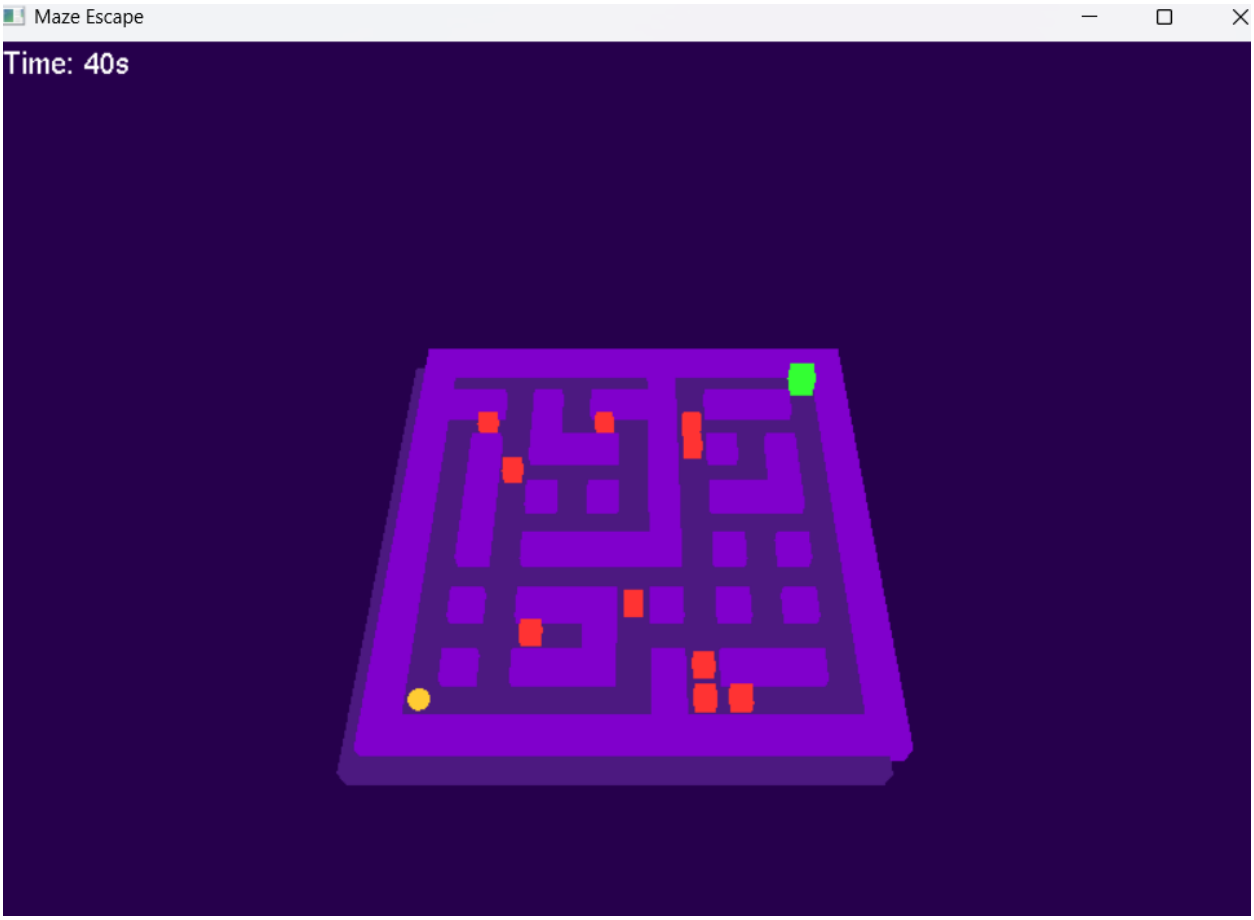


Figure 5: Level - Medium

8.4 Hard Level Maze

- Maze size: 21×21 grid with long, intricate, and twisting routes.
- Contains 20 red obstacles blocking shortcuts and adding challenge.
- Time limit: 30 seconds to reach the green exit cube.

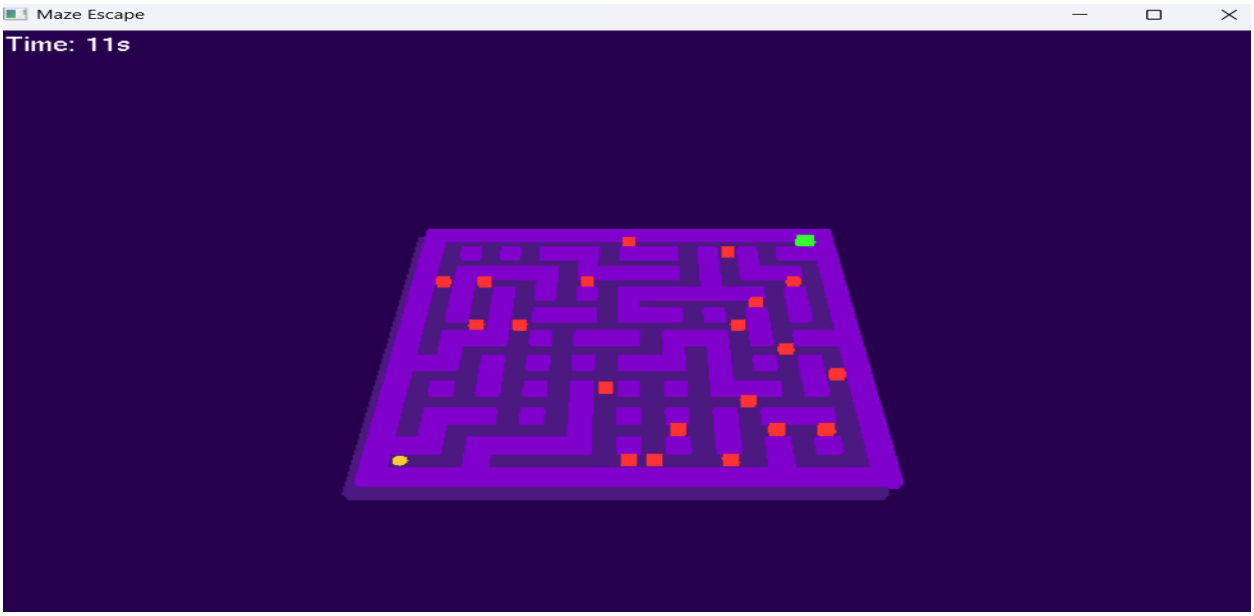


Figure 6: Level - Hard

8.5 Boundary Walls

- Thick purple walls form the outer limits of the maze.
- Prevents movement outside the maze layout.
- Requires finding and following an open passage to the exit.

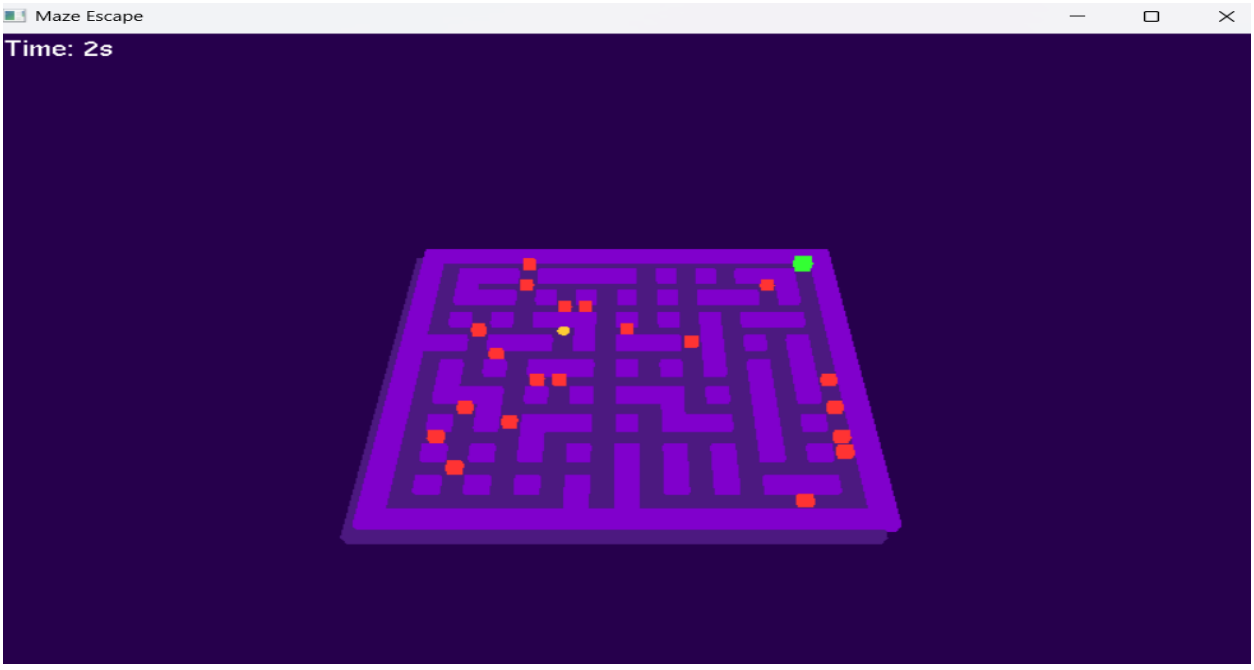


Figure 7: Boundary Obstacle Block

8.6 Internal Obstacles

- Red cube blocks placed within maze corridors.
- Cannot be crossed; player must navigate around them.
- Placement changes based on selected difficulty level.

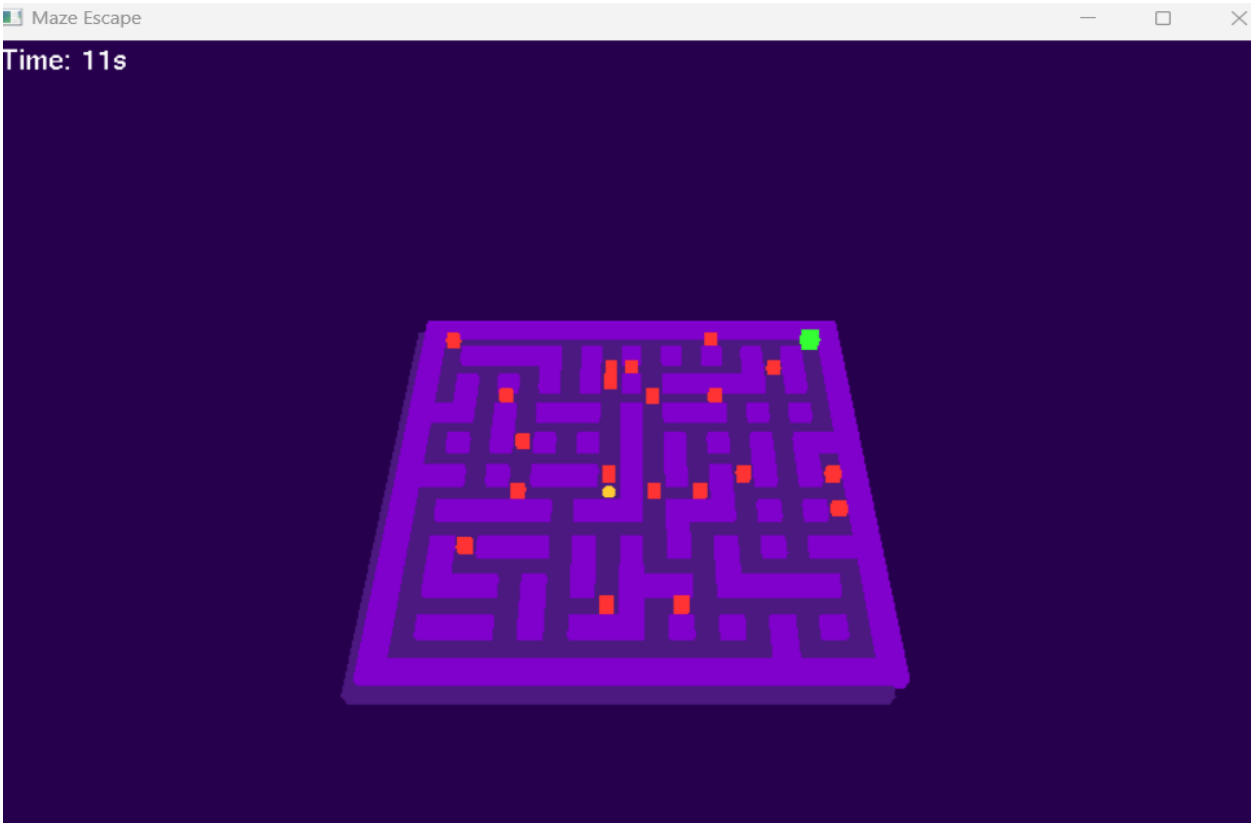


Figure 8: Internal Obstacle Block

8.7 Crash into Obstacle

- Crash Trigger: When the player touches an obstacle in the maze.
- Effect: Game immediately ends with a "crash" state.
- Redirect: Player is sent back to the Start Screen or restarts if they press R.

Figure 9: Crash Occurs

8.8 Time Expiry Condition

- Timer counts down from the set limit based on difficulty.
- At 0 seconds, the game state changes to LOST.
- Displays *"TIME'S UP! Press R to restart"* and returns to the front page on pressing R.

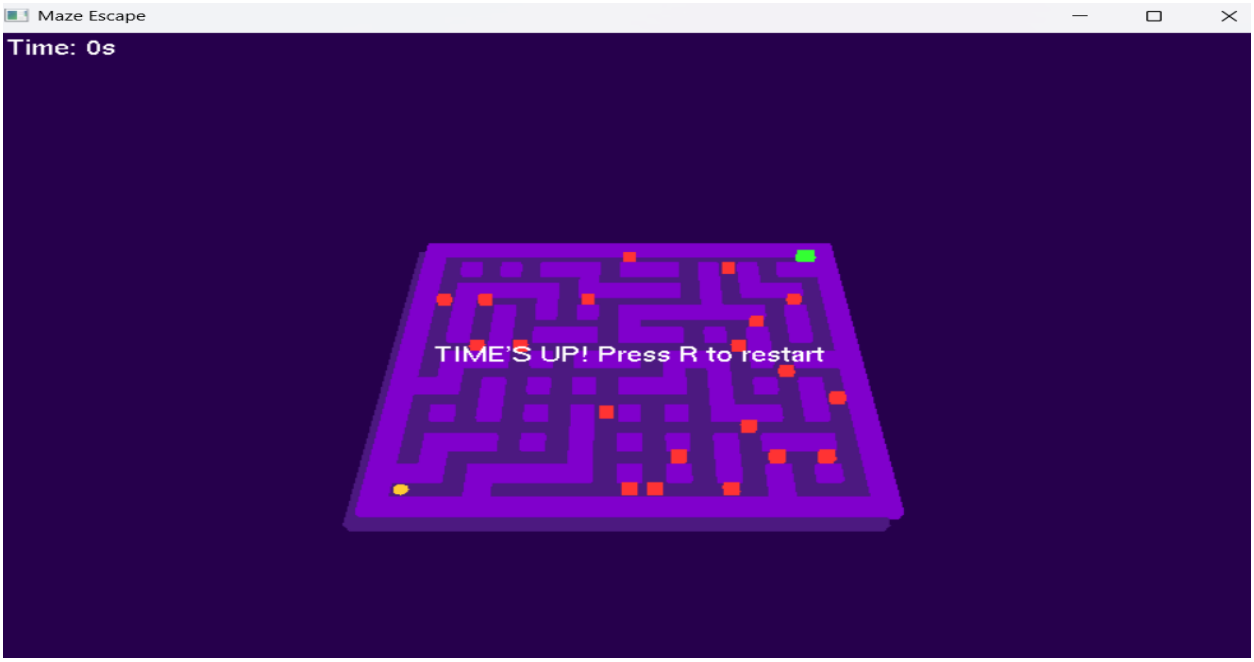


Figure 10: Time Exceeds

8.9 Clear Path Near Exit

- Exit (green cube) may sometimes have an unblocked, direct route nearby.
- No walls or obstacles obstruct the path.
- Allows for quick and easy completion if noticed.

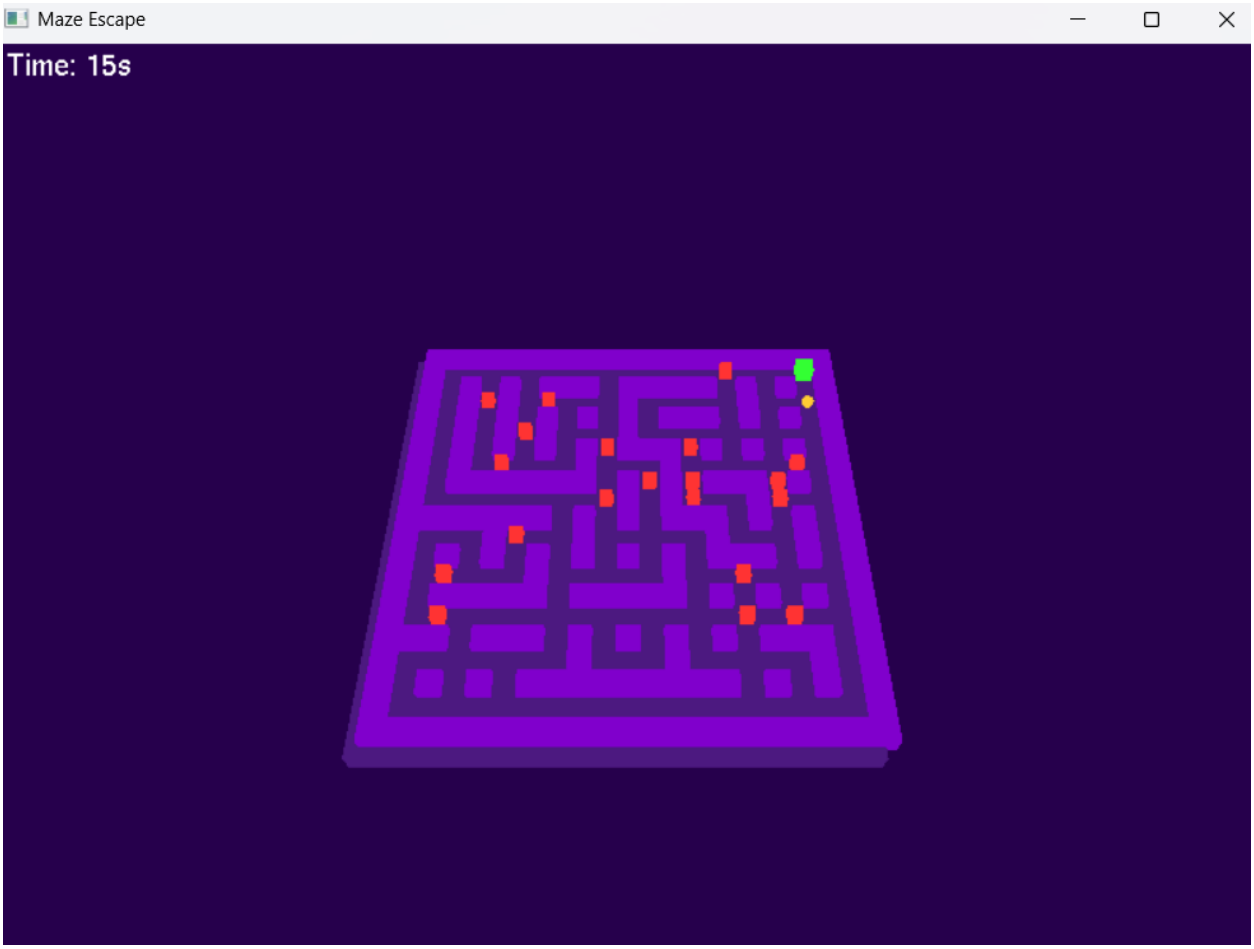


Figure 11: Near to the Exit

8.10 Winning Condition

- Player reaches the green cube to trigger WON state.
- Displays *"YOU WON! Press R to restart"*.
- Pressing R restarts the game from the front page.

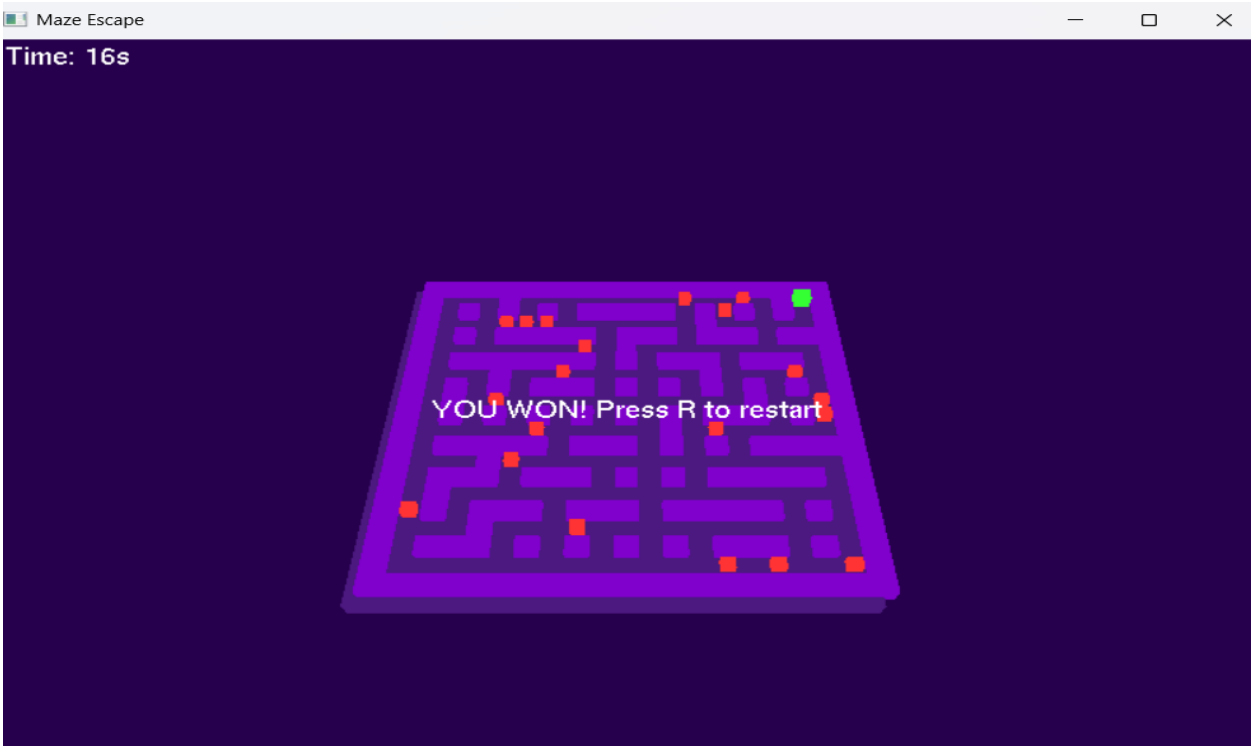


Figure 12: Winning Screen

9. Challenges Faced

9.1 Obstacle Placement Issues

One of the biggest challenges was **ensuring that obstacles did not block the only path to the exit**. Initially, randomly placed obstacles sometimes made the maze unsolvable.

Solution:

- **Path Validation**:
    - We implemented **BFS** to find the shortest path from the player to the exit.
    - Before placing obstacles, we marked all tiles along this path as **"critical"** and excluded them from obstacle placement.
- **Multiple Paths**:

    - We modified the maze generation algorithm to **create alternative routes**, ensuring that blocking one path wouldn't trap the player.
- **Strategic Obstacle Placement**:
    - Obstacles were placed only in **non-critical areas**, guaranteeing that players always had at least one viable route to the exit.

9.2 Timer Synchronization

Another challenge was **accurately tracking time** without causing lag or delays in rendering.

Solution:

- We used **clock ( )** from <ctime> to measure elapsed time precisely.
- The timer updates **every second** without interfering with the game loop.

10. Conclusion

The **Maze Escape** project successfully demonstrates:

- **Proficiency in OpenGL** for 3D rendering and transformations.
- **Effective game mechanics**, including collision detection and procedural generation.
- **Problem-solving skills** in overcoming challenges like obstacle placement and timer synchronization.

Future Enhancements:

- **Improved Graphics**: Add textures, lighting, and shadows for a more immersive experience.
- **Advanced AI**: Introduce enemy characters that chase the player.
- **Multiplayer Mode**: Allow two players to compete or cooperate.

This project serves as a **strong foundation** for further exploration in game development and computer graphics.